

The QEC Challenge

A public, automatically verified leaderboard for quantum LDPC codes

Unitary Foundation

`github.com/unitaryfoundation/qlldpc-challenge`

This talk

1. Why quantum error correction, and what makes a code good
2. Distance: the hard part, and how claims can be checked anyway
3. The Challenge: an autoresearch loop with a trusted referee

Why error correction at all

- ▶ Physical qubits err at rates $\sim 10^{-3}$ – 10^{-2} per operation; useful algorithms need 10^{-10} or better.
- ▶ Fix: encode k **logical** qubits redundantly into n **physical** qubits, measure carefully chosen parities (*syndromes*), and undo errors faster than they accumulate.
- ▶ **Threshold theorem**: below a constant physical error rate, logical errors can be suppressed exponentially in the code distance d .
- ▶ The whole game is the **overhead**: how many physical qubits per logical qubit, at a given protection level?

Stabilizer codes and the CSS recipe

A CSS code is specified by two binary parity-check matrices:

$$H_X \in \mathbb{F}_2^{r_X \times n}, \quad H_Z \in \mathbb{F}_2^{r_Z \times n}, \quad \text{with } H_X H_Z^T = 0 \pmod{2}.$$

- ▶ Each row of H_X is an X -type parity check (a stabilizer); each row of H_Z a Z -type one. The orthogonality condition makes them all commute.
- ▶ Number of logical qubits:

$$k = n - \text{rank } H_X - \text{rank } H_Z.$$

- ▶ Everything about the code is *recomputable* from (H_X, H_Z) — a fact the Challenge leans on heavily.

Errors, split in two

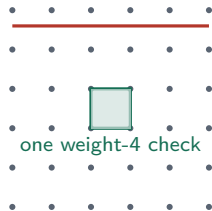
Bit flips (X) are caught by Z -checks; phase flips (Z) by X -checks. A CSS code is two classical codes stitched together at the commutation condition.

The benchmark: the surface code

- ▶ Qubits on a 2D grid; every check touches only its **4 neighbours** $\Rightarrow$ easy to build and measure.
- ▶ $d \sim \sqrt{n}$, but **one** logical qubit per patch: $k/n \rightarrow 0$ — a useful machine needs millions of physical qubits.
- ▶ Already LDPC; the problem is **2D locality**, not check weight.

Can we keep the low-weight checks, drop the locality, and get a constant rate?

a logical operator: weight d



Quantum LDPC codes, and the yardstick

LDPC = low-density parity checks: every check touches $O(1)$ qubits and every qubit sits in $O(1)$ checks — the property that makes syndrome measurement practical.

- Breakthrough (2020s): families with **constant rate and linear distance** exist. Practical frontier today: e.g. IBM's bivariate-bicycle “gross code” $[[144, 12, 12]]$ — 12 logical qubits at distance 12 from 144 data qubits, weight-6 checks.
- Headline figure of merit:

$$\text{efficiency} = \frac{k d^2}{n} \quad (\text{surface code} \approx 1, \quad \text{gross code} = 12, \quad \text{board record} = 23.81).$$

Why kd^2/n ? (Bravyi–Poulin–Terhal, 2010)

Any 2D-*local* code obeys $kd^2 \leq c n$; the surface code saturates it at ≈ 1 . So $kd^2/n \gg 1$ **quantifies how far beyond a strict 2D layout** a code performs — the record of 23.81 ($[[336, 20, 20]]$, a two-block group-algebra code on $\text{PSL}(2, 7)$) is a statement about connectivity, not just cleverness.

Distance: the hard part

A **logical operator** acts on the encoded qubits without triggering any check; d is the weight of the cheapest one — the cheapest *undetectable error* — and it enters the figure of merit *squared*.

- ▶ **Computing d exactly is NP-hard.** Every distance in every paper came from a bounded search — and bounded searches miss things.
- ▶ But any search that *finds* a logical yields an explicit **witness**: a vector anyone can check in milliseconds. It proves $d \leq \text{wt}(v)$ — trustlessly.
- ▶ Exact values ($d = v$): integer program per logical class — feasible for small codes, hopeless for large high-rate ones.

The trap

A *high* estimate from a *short* search looks like a great code. It usually means the search hasn't found the light logical yet. Distance claims need adversarial checking.

The two corroboration searches

Both only ever *exhibit* a logical of some weight — each find tightens $d \leq w$, neither proves a lower bound. They attack the claim by **independent mechanisms**:

Random information set (RIS) algebraic

- ▶ Basis of $\ker H_Z$: everything that commutes with the checks.
- ▶ Each trial: shuffle the columns, RREF the basis — random re-coordinatizations make different vectors *sparse*.
- ▶ Light rows (and pairwise sums) that are not stabilizer products are logical operators; keep the lightest.

BP+OSD decoder residuals operational

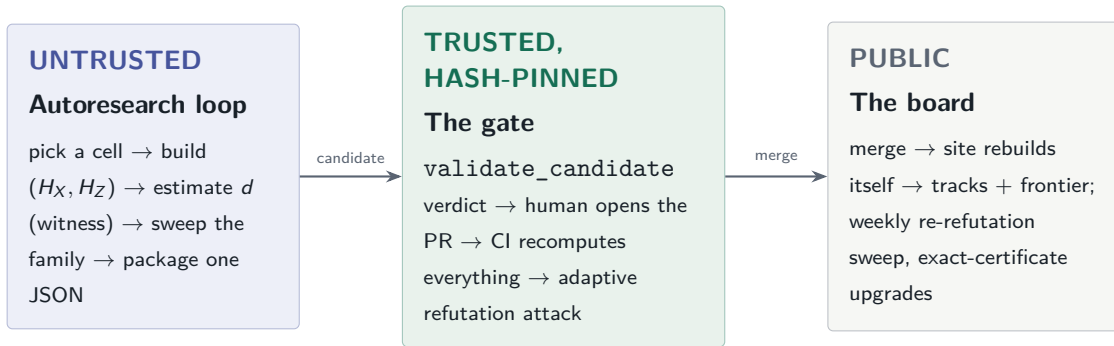
- ▶ Each trial: inject a random weight- w error e , hand its syndrome to a BP+OSD decoder, get correction $\hat{e}$.
- ▶ The residual $e + \hat{e}$ has zero syndrome by construction; if it acts on the logicals, it *is* a logical operator.
- ▶ A good decoder's failures concentrate on the *cheapest* undetectable errors — exactly what d measures.

Survives **both** at deep budgets $\Rightarrow$ corroborated ($d \leq v^*$); either finds lighter $\Rightarrow$ **refuted**, witness attached.

What the Challenge is

- ▶ A **public leaderboard** of qLDPC codes: one JSON file per code, in a git repository, with a static site on top. 49 codes today
- ▶ Every claim is **recomputed from the parity checks** — n , k , CSS commutation, check weights, distance witnesses, 2D-locality radius. Nothing is trusted from the submitter.
- ▶ Codes compete on **Pareto frontiers** inside track cells (check-weight class $\times$ locality class), never on a single score.
- ▶ Designed so that **LLM research agents** can do the searching — with a referee they cannot talk their way past.

Three zones, one direction of trust



The one rule (from the agent manual, `AUTORESEARCH.md`)

No code is a find until `validate_candidate` returns `passed: true`. You may write and run any code you like — but you do not get to decide whether a code is good.

The CI gauntlet, per submission

1. **Scope:** a PR adding a code may not touch workflows, the schema, or `verify/` — a submission cannot change its own judge. One new code per PR.
2. **Integrity pin:** SHA-256 of the whole trusted stack against a committed manifest; any drift fails the build.
3. **Self-tests:** the verifier must still *reject* known-bad submissions and known over-claims.
4. **Recomputation:** n , k , CSS, weights, witnesses — recomputed by the *base branch's* verifier. Locality is **measured** from a submitted layout certificate, never declared; no honest layout $\Rightarrow$ the code competes as *unrestricted*.
5. **Adaptive refutation:** two independent searches attack the distance claim, budget scaling with size $(2500 + 40n)$ trials). A claim that advances its cell's frontier triggers the deep tier: a $\sim 150\times$ -deeper accelerated search plus a Python canary seed (three full Python seeds if the accelerator is unavailable). One lighter logical found $\Rightarrow$ PR fails, witness printed.
6. **Authorship:** the PR author must be a listed author of the code.

Distance claims are tiered by how they were earned

Tier	Meaning	Earned by
$d \leq v^*$	corroborated	witness checked trustlessly; two independent searches (RIS, BP+OSD) fail to find anything lighter
$d = v$	exact	server MILP certificate committed to certs/
refuted	claim killed	any search finding a lighter logical, witness attached

- ▶ 29 of 49 board codes hold an exact certificate.
- ▶ The other defense is time: a **weekly random-seed sweep** re-attacks every entry on the board, forever.

What it is for

- ▶ A **shared, machine-checked record** of the best qLDPC codes under each constraint — parameters that today live scattered across papers.
- ▶ A **benchmark for AI research agents**: a tool-agnostic, pure-numpy starter kit plus an operating manual (`AUTORESEARCH.md`) any model can be pointed at.
- ▶ **Shared field notes**: campaigns end by distilling hard-won lessons into the manual — every newcomer, human or agent, starts where the last one stopped.
- ▶ A template for **verification-first open science**: untrusted exploration, a pinned referee, claims that decay with evidence instead of persisting as folklore.

Beat the frontier: github.com/unitaryfoundation/qldpc-challenge

Thanks!

Live board, starter kit, and the full verification stack:

`github.com/unitaryfoundation/qldpc-challenge`

`unitaryfoundation.github.io/qldpc-challenge`